

Laboratorio 3 · notas del docente

Interpolación con un agente de RL · 90 minutos

Resultados medidos el 6 de octubre de 2026 en un MacBook Air (Apple M5, 24 GB) con macOS 26.5.2 y Python 3.13.13, semilla 1 salvo que se diga otra cosa. Las cifras son idénticas a las de la primera medición (Linux, Python 3.12); los tiempos son los de este Mac. Todo se reproduce con `docs/curso/lab03/experimentos.py` y `graficar.py`. Usa el proyecto 07; en el arco estaba como lab 5 y se adelantó porque la clase del 1 de octubre cubrió Taylor y Lagrange. El laboratorio de Taylor (proyecto 03) sigue pendiente.

1 Plan de la sesión

min	bloque	modo
0-10	descargar el ZIP de hoy, instalar 07 con <code>matplotlib, --no-tui</code> (8 s)	guiado
10-25	basales a mano (a)-(d), sin IA; revisar con <code>basales</code>	parejas, papel
25-40	<code>graficar.py</code> : ajustar <code>PUNTOS</code> , <code>RANGO</code> , <code>NODOS</code> ; PIB (e)-(g)	parejas
40-55	agente con animación, dos semillas, informe (h)-(i)	parejas
55-70	familias, <code>solo_equi</code> , <code>sin_runge</code> , recompensa (j)-(l)	parejas + plenaria corta
70-85	dibujar, grafo (m)-(n); <code>mi_grafo</code> (o)	parejas + plenaria
85-90	cierre y entrega	expositivo

Las ideas de la sesión son tres: la suma de valores absolutos de los basales es lo que amplifica los errores (de (c) a (g) y a (h)); lo que el agente aprende depende de los problemas que ve ((k)); y el verificador solo sabe lo que dicen las flechas ((n) y (o)). Si se va el tiempo, cortar (i), (k) y la tarea opcional, en ese orden. No cortar (c), (g) ni `mi_grafo`.

2 Tiempos medidos

orden	tiempo
<code>python -m interprl --no-tui --seed 1</code>	8 s (fase 1: 7,5-8,2 s; fase 2: 0,1-0,2 s)
<code>python -m interprl --seed 1</code> (animación)	2,5 min (2 min 28 s y 2 min 39 s)
<code>basales</code> , <code>dibujar</code> , <code>grafo</code> , <code>mi_grafo</code>	< 0,1 s
<code>familias</code>	0,8 s
<code>hitos --seed N</code>	8 s (fases 1 y 2 completas)
<code>solo_equi</code> , <code>sin_runge</code>	6,6 s (dos entrenamientos de 2000 gen.)
<code>recompensa</code>	10 s (tres entrenamientos)
<code>graficar.py</code>	0,4 s; la primera vez, 6 s (caché de fuentes de <code>matplotlib</code>)
<code>search --n 4</code>	1 s

Son dos tandas seguidas que coinciden entre sí. La primera tanda, justo después de instalar, fue más lenta: `familias` 12 s y los entrenamientos de `experimentos.py` alrededor de 1,5 veces más. De los 2,5 min de la animación, el cálculo son unos 8 s; el resto son las pausas fijas de la animación y el dibujo. En la primera medición (Linux, Python 3.12) los tiempos eran algo más del

doble que estos (17 s para --no-tui). En los PCs del laboratorio no se ha medido: contar con que sea más lento que aquí. Avisar que --no-tui no está colgado.

3 Respuestas y cifras

- (a) $L_0 = t(t-1)/2$, $L_1 = 1 - t^2$, $L_2 = t(t+1)/2$.
- (b) $L_0 = (t-1)(t-3)/3$, $L_1 = -t(t-3)/2$, $L_2 = t(t-1)/6$. El error típico es el signo de L_1 : el denominador es $(1-0)(1-3) = -2$.
- (c) Lo que imprime `basales`:

nodos	t	L_0	L_1	L_2	suma	suma de $ L_k $
-1, 0, 1	-1/2	3/8	3/4	-1/8	1	5/4
-1, 0, 1	1/2	-1/8	3/4	3/8	1	5/4
0, 1, 3	2	-1/3	1	1/3	1	5/3
-1, 0, 1	2	1	-3	3	1	7

La constante de Lebesgue de -1, 0, 1 en $[-1, 1]$ es exactamente 5/4, alcanzada en $t = \pm 1/2$: el número que calcularon a mano. Vale la pena decirlo en voz alta.

- (d) Una versión aceptable, que es también la solución de `mi_grafo` (7 pasos; el agente la ordena en la generación 5):

clave	afirmación	deps
H1	los nodos son distintos	—
DEF	$L_k(t) = \prod_{j \neq k} (t - x_j) / (x_k - x_j)$, grado n	H1
DELTA	$L_k(x_j) = 1$ si $j = k$, 0 si no	DEF
S	$S(t) = \sum_k L_k(t)$ tiene grado $\leq n$	DEF
SNODOS	$S(x_j) = 1$ para todo j	S, DELTA
UNIQ	un polinomio de grado $\leq n$ con $n+1$ ceros es nulo	H1
QED	$S - 1$ tiene grado $\leq n$ y $n+1$ ceros, luego $S \equiv 1$	S, SNODOS, UNIQ

Distractor natural: « S vale 1 en los $n+1$ nodos, luego $S \equiv 1$ » (le falta el grado $\leq n$; la flecha de S a QED es la que lo descarta).

- (e) Vietnam: $P(2013) = 216,3$ frente a 213,7 real (+1,2%); $P(2015) = 247,1$ frente a 239,3 (+3,3%). La serie es suave entre los nodos y los basales en $\pm 1/2$ están entre $-1/8$ y $3/4$: no amplifican. 2015 es peor porque el crecimiento en dólares casi se detiene ese año ($233,5 \rightarrow 239,3$).
- (f) Colombia: $P(2020) = 317,8$ frente a 270,3 real (+47,5; +17,6%). El polinomio solo sabe lo que dicen los nodos, y la caída de 2020 no dejó rastro en 2018, 2019 ni 2021. La fórmula del error necesita f''' acotada y conocida; con datos no hay f , y un choque de un año hace que cualquier f razonable tenga f''' enorme. La fórmula dice cuándo confiar, no cuánto se equivoca uno con datos reales.
- (g) China: $P(2022) = 20\,179,9$ frente a 17 881,8 (+12,9%); $P(2023) = 22\,932,0$ frente a 17 794,8 (+28,9%). Basales en $t = 3/2$: $3/8$, $-5/4$, $15/8$ (suma de absolutos 3,5); en $t = 2$: 1, -3 , 3 (suma 7). Un 1% del dato de 2021 (178,2) se multiplica por $L_2(2) = 3$: la estimación de 2023 se mueve 534,6. Interpolan no es pronosticar.

Hitos (`experimentos.py hitos --seed N`; idénticos a los de la animación con la misma semilla):

hito	seed 1	seed 2
primera aproximación correcta	80	12
descubrió el criterio de parada	80	111
su regla coincide con la teoría (8 estados, estable 100 gen.)	526	542
prefiere $\gamma = 1$ de forma estable	726	572
primera demostración completa	186	215
primera demostración mínima	463	368

Con seed 2 aparece además « $\geq 95\%$ de aciertos» en la 2490; con seed 1 no. Al final: seed 1, 89% de éxito en las últimas 100, 2187 correctas, 792 prematuras, 21 agotadas; seed 2, 80%, 2186, 782, 32.

- (h) Seed 1: $Q(\gamma = 1) = -2,8$ (2584 usos) frente a $Q(\gamma = 0) = -13,4$ (110 usos). $Q(\gamma)$ es el retorno medio de una partida con esa familia; con equiespaciados las funciones tipo Runge acaban en entrega prematura o agotadas. Tabla de Λ del informe: con $n = 16$, equiespaciados 934,5, Chebyshev 2,7. Es lo mismo que la última fila de (c): basales grandes amplifican, y con equiespaciados crecen como 2^n .
- (i) Cuando los interpolantes convergen geométricamente, $|p_n - p_{n-1}|$ es del orden del error de p_{n-1} y acota bien el de p_n . Engaña cuando todavía no hay convergencia o el error se estanca. Costo: 792 entregas prematuras de 3000 con seed 1. Es el problema de Colombia: lo que no se ve en las muestras no existe para el método.
- (j) familias (120 problemas, máximo 40 nodos): Runge $1/(1 + ax^2)$, equiespaciados 0/31, Chebyshev 31/31 con 29,2 nodos; polo $1/(x - c)$, 27/29 (22,5) frente a 29/29 (16,5); raíz, 17/17 (14,9) frente a 17/17 (11,2); enteras, 43/43 (10,4) frente a 43/43 (9,5). **solo_equi** (2000 gen.): éxito $85\% \rightarrow 25\%$, prematuras $653 \rightarrow 1505$, agotadas $14 \rightarrow 7$. Con Runge cada nodo equiespaciado sube el error: el progreso es negativo en cada paso y seguir cuesta más que el -10 de entregar mal. Rendirse es lo racional con esos nodos.
- (k) **sin_runge**: éxito 94% , $Q(\gamma)$ entre $+0,7$ y $+7,5$, preferida $\gamma = 0,75$. Ya no hay una familia que gane con claridad (las diferencias están dentro del ruido del bandido, $\alpha = 0,05$). Lo aprendido en la parte 4 no es «Chebyshev es mejor» como teorema, sino «con estos problemas, Chebyshev da más puntos». El teorema es sobre los nodos; la preferencia del agente aparece solo cuando los problemas la hacen importar. La distribución de problemas es parte de la especificación, como la recompensa.
- (l) Original: 85% , 1333/653/14, $\gamma = 1$. «Entregar mal cuesta 0»: 0% , 1 correcta y 1999 prematuras: entrega siempre en el primer paso. Una partida que trabaja hace unos 18 nodos (-18), el progreso telescopa a $\log_{10}(\text{error inicial/error final}) \approx 5$ y gana $+10$: unos -3 . Entregar ya da 0. Es el (f) del lab 1. «Cada nodo gratis»: 87% , $\gamma = 1$, casi nada cambia: el descuento $0,95$ hace que el $+10$ valga más cuanto antes y, pasado el error de máquina, añadir nodos no da progreso. También probado **--entregar-mal -1 --nodo 0**: 85% , $\gamma = 1$.
- (m) Si los $n + 1$ nodos se juntan en x_0 , las condiciones pasan a ser $f(x_0), f'(x_0), \dots, f^{(n)}(x_0)$ (Hermite en el límite), el interpolante es el polinomio de Taylor y $w(x) \rightarrow (x - x_0)^{n+1}$. Taylor es el caso en que toda la información está en un punto: la idea con la que se abrió la clase del 1 de octubre.
- (n) Si $x = x_i$, $w(x) = 0$ y g ni siquiera está definida; **FIX** trata ese caso aparte (ambos lados son 0). Sin la flecha **FIX** $\rightarrow$ **AUX**, la demostración de 12 pasos que acepta el verificador divide por cero sin decirlo. Responsable: quien escribió las **deps**.
- (o) Libre. Distractores buenos: el del grado (arriba); «cada L_k está entre 0 y 1, luego la suma es $\leq n + 1$ » (falso: la fila $t = 2$ de (c)); « $S = 1$ porque los basales son una partición de la unidad» (circular).

Párrafo final. Que mencionen: rellenar datos entre muestras de algo suave (sirve); no sirve para choques que no tocan los nodos (Colombia) ni para extrapolar (China); más nodos equiespaciados no arreglan nada (Runge). Bonus si conectan con Λ .

Tarea opcional. Equiespaciados con 5 nodos: $\Lambda = 2,2078$. Interiores en $\pm 0,7$: $\Lambda = 1,7727$. `search --n 4` (1 s) encuentra $\pm 0,620911$ con $\Lambda = 1,55949$, por debajo del Chebyshev extendido (1,57017).

4 Trampas

- **El ZIP tiene que ser de hoy.** Si lo descargaron antes, no trae `docs/curso/lab03`.
- `matplotlib` va en la misma línea de instalación. Si falta, `graficar.py` dice cómo instalarlo.
- En `basales` los nodos van con `=`: `--nodos=-1,0,1`. Sin `=`, el `-1` parece una opción.
- `graficar.py` se edita. Si alguien lo rompe, se recupera del ZIP o con `git checkout`.
- Con `PUNTOS = [..., 2]` y `RANGO = None`, la línea en $t = 2$ existe pero cae fuera del eje: es intencional, es el ajuste 2.
- El PIB está en dólares corrientes: mezcla producción, inflación y tasa de cambio. La caída de Colombia entre 2014 y 2015 ($381,2 \rightarrow 293,5$) es sobre todo devaluación del peso.
- `pib_tres_paises.csv` está congelado (Banco Mundial vía `datasets/gdp`, CC BY 4.0, hasta 2023). `graficar.py --reconstruir` lo regenera.
- `mi_grafo.py` sin tocar da un error a propósito: ■ sin deps. Es el primer mensaje que ven.